Session Calendar — 21 Weeks × 105 Live Sessions

Schedule source of truth. Follow the modules listed here in order. Classroom pacing follows this calendar.

Setting	Value
Live class	**1 hour 15 minutes**
Days	Mon–Fri (5 sessions / week)
Length	**21 weeks** (20 + bridge week 12A for HTML & CSS)
Total live sessions	**105** (S1–S100 + HTML & CSS sessions F1–F9, which replace the old S56–S59)
Live hours	105 × 1.25h = **131.25 hours**
Homework	~45–90 min after most sessions

Compressed option: 16 weeks is *not* recommended for this syllabus at 75 minutes/day. Use only with Saturday labs or fewer React/DRF outcomes.

Rule: One live session = **one primary focus** (one concept block + one mini-lab). Do not schedule two full modules in one 75-minute class.

Engagement rule: Start from a lab ticket (`15_Lab_Assets/` , or `GIT/labs/` , `FRONTEND/HTML_CSS/labs/` , `FRONTEND/labs/` , `FRONTEND/REACT/labs/`), then open module notes for Definition/Purpose as needed.

Biweekly Assessment: After every even week (Weeks 2, 4, … 20), complete the matching paper from `16_Assessments/` (BW01–BW10) as homework or Friday wrap-up. Graded papers live only under `16_Assessments/` — not inside modules.

Shared topics (repo root): `GIT/` · `MYSQL/` · `FRONTEND/` (includes `FRONTEND/REACT/`) — scheduled here, authored outside this folder.

Typical class shape (timing lives here — not repeated in every module):

Minutes	Block
5	Open lab kit
25	Concept demo / notes
35	Lab (starter/broken)
10	Exit + homework

How to read the table

Column	Meaning
S#	Global session number (01–100)

Column	Meaning
Focus	What you practice *in class*
Module file	Notes to open
Homework	Must finish before next session

Module file: click the link (or keep **same** = previous session's file).

Phase A — Foundations & Core Python (Sessions 01–30)

S#	Wk	Focus	Module file	Homework
01	1	Computer, OS, files, terminal basics	01_Foundations/01_Computer_and_Programming_Foundations.md	Workspace folders
02	1	Client–server + roles; VS Code tour	same	Roadmap note
03	1	Install Python 3; verify PATH	02_Core_Python/01_Python_Setup_and_First_Programs.md	Setup proof screenshot
04	1	REPL vs script; first programs	same	`hello.py` on GitHub account created
05	1	Keywords, identifiers; lab catch-up	same	Flash keywords
06	2	Variables & types	02_Core_Python/02_Variables_Datatypes_Operators_IO.md	Type drills
07	2	Operators	same	Calculator v1
08	2	Input/output & f-strings	same	Canteen calculator + f-string receipt
09	2	Casting bugs lab	same	Fix 5 casting mistakes
10	2	Mini: CLI calculator polish	same	Push repo
11	3	`if` / `elif` / `else`	02_Core_Python/03_Flow_Control.md	Grade calculator
12	3	`for` / `range`	same	Tables & patterns
13	3	`while`, break/continue	same	Number guessing
14	3	Nested menus	same	Start ATM menu
15	3	ATM menu lab day	same	Finish ATM + push
16	4	Strings: index, slice, immutability	02_Core_Python/04_Strings.md	Slice drills
17	4	String methods & formatting	same	Email/phone parse
18	4	Lists CRUD & methods	02_Core_Python/05_Lists.md	Marks list

S#	Wk	Focus	Module file	Homework
19	4	Nested lists + comprehension intro	same	Filter lab
20	4	Marks Manager mini-project	same	Push CLI app
21	5	Tuples & unpacking	02_Core_Python/06_Tuples_and_Sets.md	Tuple drills
22	5	Sets & uniqueness	same	Dedup tool
23	5	Dictionaries CRUD	02_Core_Python/07_Dictionaries.md	Contact lookups
24	5	Nested dict / JSON-shaped data	same	Contact book start
25	5	Contact book lab	same	Push contact book
26	6	When to use list/tuple/set/dict	02_Core_Python/08_Data_Structures_Overview.md	Decision drills
27	6	Stack & queue with list	same	Undo demo
28	6	Functions: `def`, return, params	03_Advanced_Python/01_Functions_Decorators_Generators.md	Refactor ATM funcs
29	6	`*args` / `**kwargs`, scope	same	Start `ksr_utils` validators + formatters
30	6	Lambda + `map` / `filter` / `reduce` + decorator intro	same	`transforms` + `@timing` (no yield)

Phase B — Advanced Python & Git (Sessions 31–45)

S#	Wk	Focus	Module file	Homework
31	7	Generators vs lists	03_Advanced_Python/01_Functions_Decorators_Generators.md	Yield lab (+ optional `generators.py`)
32	7	Utils package mini-project	same	Push `ksr_utils` (core; yield is stretch)
33	7	Modules & imports	03_Advanced_Python/02_Modules_and_Packages.md	Import drills
34	7	Packages, venv, pip	same	venv checklist
35	7	Stdlib: json/os/datetime lab	same	JSON file tool
36	8	Class / object / `__init__`	03_Advanced_Python/03_OOP_Part1_Classes_Encapsulation.md	Student class
37	8	Encapsulation + methods	Part 1 same	BankAccount

S#	Wk	Focus	Module file	Homework
38	8	Inheritance + `super()`	03_Advanced_Python/03_OOP_Part2_Inheritance.md	Employee hierarchy
39	8	Polymorphism lab	03_Advanced_Python/03_OOP_Part3_Polymorphism_Project.md	Push OOP app (BankAccount CLI)
40	8	ABC / interfaces	03_Advanced_Python/03_OOP_Part4_Abstract_Classes_Interfaces.md	Campus Wallet lab
41	9	Exceptions try/except + custom	03_Advanced_Python/04_Exception_Handling.md	Safe input lab
42	9	Files + `with` + CSV/JSON	03_Advanced_Python/05_File_Handling.md	CSV append
43	9	Logging levels	03_Advanced_Python/06_Logging_Module.md	Logger in expense app
44	9	Git Session 1: first GitHub push	../GIT/01_Git_Session1.md	KSR_MART on GitHub
45	9	Git Session 2: daily loop + clone	../GIT/02_Git_Session2.md	Hygiene lab; homework Session 3 · 4 · 5

Phase C — MySQL & Python DB (Sessions 46–55)

External dependency: repo `MYSQL/` (and `FRONTEND/` / `FRONTEND/REACT/` on later weeks).

S#	Wk	Focus	Module / lab	Homework
46	10	Why DB; install/connect	MYSQL README + 00_MySQL_Setup + FS bridge	`training_db` created
47	10	Tables, PK; DDL CREATE	MYSQL/01–02	3 tables
48	10	DML INSERT/UPDATE/DELETE	MYSQL/02	Seed data
49	10	DQL WHERE/ORDER/LIMIT	MYSQL/03 + labs seed	Report queries
50	10	FS bridge concepts + ER sketch	05_Database_MySQL/01_MySQL_for_Full_Stack_Developers.md	Library ER
51	11	Modeling PK/FK	MYSQL/06	Related tables
52	11	INNER JOIN	MYSQL/07	Join drills
53	11	LEFT JOIN + multi-table	MYSQL/07	Customers w/o orders
54	11	TCL + indexes lite	MYSQL/09	Rollback demo
55	11	Python↔MySQL CRUD	03_Advanced_Python/07_Database_Programmi	Parameterized CRUD push

S#	Wk	Focus	Module / lab	Homework
			ng_Bridge.md	

Phase D — Frontend (Sessions F1–F9, 60–62)

HTML & CSS (F1–F9): nine sessions from the HTML & CSS learning path, one AcmeHR page per session. They are numbered F1–F9 so that every other session keeps its number; week **12A** is the extra bridge week.

S#	Wk	Focus	Module	Homework
F1	12	How the web works & first page	Lesson_01 · lab	Welcome page served locally
F2	12	Content & first CSS	Lesson_02 · lab	Leave policy page
F3	12	Page structure & box model	Lesson_03 · lab	Employee profile
F4	12	Images, navigation & Flexbox	Lesson_04 · lab	Employee directory
F5	12	Grid & positioning	Lesson_05 · lab	HR dashboard
F6	12A	Forms (GET/POST, what the server receives)	Lesson_06 · lab	Apply-for-leave form
F7	12A	Tables	Lesson_07 · lab	Attendance report
F8	12A	Responsive design	Lesson_08 · lab	Dashboard on a phone
F9	12A	Polish, accessibility & final project	Lesson_09 · lab	Final project: AcmeHR portal
60	12A	JS variables, DOM, events	../FRONTEND/03_JavaScript_DOM_Forms.md	Click counter
61	13	Form validation	same	Validate contact
62	13	`fetch` JSON overview	same	Portfolio page push

Phase E — Django (Sessions 63–80)

S#	Wk	Focus	Module	Homework
63	13	Django install, project, app	07_Django/01_Django_Introduction_and_Setup.md	runserver proof
64	13	Settings + MySQL connect	same	migrate
65	13	URLs + FBV hello	07_Django/02_URLs_Views_Templates_Static.md	2 pages
66	14	Templates + inheritance	same	base.html
67	14	Static CSS	same	Styled pages
68	14	Models + migrations	07_Django/03_Models_ORM_Admin.md	Product model

S#	Wk	Focus	Module	Homework
69	14	ORM CRUD + admin	same	Admin seed
70	14	Forms / ModelForm	07_Django/04_Forms_Sessions_Auth.md	Create form
71	15	Sessions + auth register/login	same	Auth works
72	15	`login_required` + owner filter	same	Protected CRUD
73	15	Security: CSRF/XSS/env	07_Django/05_Security_and_Deployment.md	`.env` + dotenv
74	15	`STORAGES`, collectstatic, `check --deploy`	same	Checklist started
75	15	**Lab 06** CRUD/auth consolidation	07_Django/06_Lab_CRUD_Auth_Consolidation.md	Feature freeze list
76	16	**Lab 06** continue + peer break-auth	same	Must-pass items
77	16	**Lab 06** demos	same	Deliverables bundle
78	16	**Lab 07** deploy debug	07_Django/07_Lab_Deploy_Capstone_Debug.md	check --deploy green
79	16	**Lab 07** PaaS attempt	same	README env table
80	16	Capstone M0–M1 shell	Capstone_CampusCart_One_Pager.md	Auth shell push

Phase F — APIs, React, Quality, Capstone (Sessions 81–100)

S#	Wk	Focus	Module	Homework
81	17	Flask routes + JSON lite	08_Flask/01_Flask_Overview.md	Notes API stub
82	17	REST + status codes + Postman	09_APIs_REST/01_APIs_JSON_Postman_DRF.md	Collection v1
83	17	DRF serializer + viewset	same	List/create works
84	17	DRF URLs + validation errors	same	Postman CRUD
85	17	`requests` client script	same	Push API repo
86	18	React Vite + first component	../FRONTEND/REACT/01_React_Fundamentals.md	Hello card
87	18	Props + state	same	Counter + list
88	18	Forms + lists/keys	same	Controlled form
89	18	useEffect + Router	../FRONTEND/REACT/02_React_Hooks_Routin	Two routes

S#	Wk	Focus	Module	Homework
			g_API_Deploy.md	
90	18	Fetch Django API + CORS	same	Product list UI
91	19	Debugging + logging habits	11_Testing_Clean_Code/01_Testing_Debugging_Clean_Code.md	Debugger lab
92	19	unittest/pytest basics	same	3 tests
93	19	Capstone M2–M3 features	Capstone one-pager	Orders flow
94	19	Capstone polish + a11y pass	Deliverables + one-pager	Screenshots
95	19	Deploy capstone / Lab 07 catch-up	Capstone one-pager	Live or staging URL
96	20	Resume + LinkedIn + GitHub pins	13_Placement/01_Career_Resume_Interview.md	Resume v1
97	20	Mock interview (Python+SQL)	same	Feedback notes
98	20	Mock interview (Django+projects)	same	STAR stories
99	20	Capstone defense rehearsals	Capstone	10-min script
100	20	Demo day + placement wrap	Placement	Applications started

AI Integration (12_AI_Integration): optional after S95 — never displaces S93–S100.

Optional Deep Dive (08_Professional_Python_Deep_Dive): after S45 or as Saturday enrichment — not required for Django path.

If the batch falls behind

Slip React/DRF sessions, not Django Lab 06/07 or SQL joins. Those are placement-critical.